

Received 17 September 2025, accepted 8 October 2025, date of publication 15 October 2025, date of current version 21 October 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3622091

RESEARCH ARTICLE

Reinforcement Learning-Based Footstep Control for Humanoid Robots on Complex Terrain

WILLIAM SULIMAN¹, EGOR DAVYDENKO¹, EKATERINA CHAIKOVSKAIA¹,
AND ROMAN GORBACHEV¹

Moscow Institute of Physics and Technology (MIPT), 141701 Dolgoprudny, Russia

Corresponding author: William Suliman (suliman.vil@phystech.edu)

This work was supported by the Ministry of Economic Development of the Russian Federation under Agreement with MIPT No. 139-15-2025-013, dated June 20, 2025, IGR 000000C313925P4B0002.

ABSTRACT We propose a Reinforcement Learning (RL)-based footstep planning framework that enables a bipedal robot to achieve stable locomotion over challenging terrain. At its core, a learned step-following policy executes footstep commands with varying positions, orientations, and heights. These commands are generated by a heuristic planner that processes user-defined velocity and orientation inputs alongside a local height field to adaptively adjust step placement and height. By leveraging two-step foresight commands for each leg, the controller ensures safe and adaptive locomotion in dynamic environments. A comparative study of different foresight horizons (one-step, two-step, and three-step) demonstrates that the two-step controller provides the best trade-off between performance and complexity. Critically, our method plans footsteps without relying on dynamic models, making it simpler to implement and deploy. We show through comparisons with a model-based approach that it achieves competitive performance even without explicit dynamics information. To handle non-flat terrain, our method adopts a modular architecture that decouples perception from control, in contrast to end-to-end approaches that map perception inputs directly to motor commands. Perceptual information is processed into footstep commands, which are then provided to the controller. This design improves interpretability, simplifies debugging, and increases safety by ensuring the low-level policy consistently receives physically feasible commands. Trained in simulation to navigate environments with randomly placed obstacles, the resulting policy demonstrates robust locomotion across diverse terrains. The robot can overcome obstacles up to 50% of its leg length while accurately tracking speeds of up to 0.8 m/s, as validated in simulation on Bruce, a kid-sized humanoid robot platform.

INDEX TERMS Bipedal locomotion, footstep planning, reinforcement learning.

I. INTRODUCTION

Achieving smooth and stable walking in bipedal robots remains a fundamental challenge in robotics. These robots aim to replicate human locomotion and are envisioned for use in diverse applications such as home assistance, healthcare, and exploration in hazardous environments. However, maintaining balance and stability across varying walking speeds and uneven terrain remains a difficult task due to the complex dynamics involved.

In this work, we present a novel Reinforcement Learning (RL)-based controller designed for robust base-velocity

tracking in dynamic environments (Fig. 2). Our approach enables the robot to adapt its footstep poses—as needed—including position, orientation, and height, in real time, allowing stable and precise locomotion over terrain with randomly distributed gaps, obstacles, and height variations (Fig. 1). In addition, robustness to external disturbances in the form of linear pushes applied to the robot's base was evaluated, demonstrating that the policy achieves dynamic balance during walking.

The proposed method aligns with classical navigation strategies in which a path is planned to guide the robot's base position toward a target. By tracking the desired base velocity, the robot is able to follow this path. Simultaneously, our system allows the robot to adjust its foot placement to account

The associate editor coordinating the review of this manuscript and approving it for publication was Jiachen Yang¹.

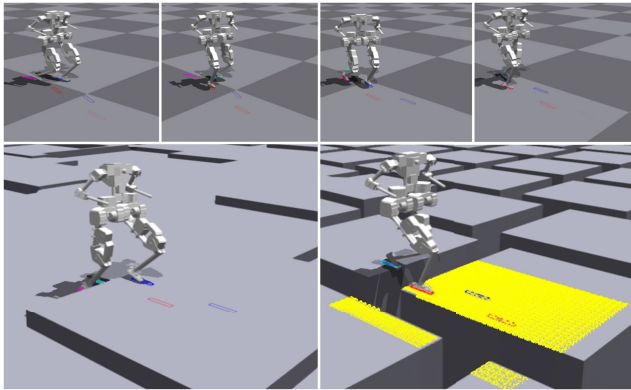


FIGURE 1. The proposed learning-based controller enables stable locomotion over challenging terrain. The policy functions as a step-following controller, accurately placing feet on randomly placed steps with varying positions and orientations. Step height and position are adaptively adjusted by a heuristic planner that processes user-defined velocity and orientation commands alongside a local height field. The simulated robot successfully navigates terrain with gaps and obstacles up to 50% of its leg length, achieving walking speeds of 0.8 m/s.

for challenging terrain conditions, such as stairs, gaps, and uneven step heights.

Unlike traditional model-based footstep planning approaches, our method is model-free and relies on a heuristic-based step command estimator. This estimator interprets user-defined velocity and orientation commands and, when navigating uneven terrain, incorporates local height-field information, which in this work is obtained directly from the simulation environment. It predicts the next two footstep targets for each leg, along with smoothed base orientation and velocity trajectories. These high-level motion plans are then passed to the RL-based controller, which outputs the corresponding desired joint positions.

The proposed framework adopts a modular architecture that explicitly separates perception and control. This design improves interpretability, facilitates deployment, and increases safety by ensuring the low-level policy consistently receives physically feasible commands.

The key contributions of this work are as follows:

- **Commanded Footstep Pose Tracking:** We propose a model-free RL controller that explicitly tracks commanded footstep poses (position, orientation, height), enabling accurate path following and obstacle avoidance. Unlike prior model-based or fixed-step methods, our controller handles varying step lengths and demonstrates performance comparable to a model-based baseline.
- **Two-Step Foresight Control:** The controller anticipates future contacts using a two-step foresight strategy, improving balance on structured terrain such as gaps and discrete steps. A comparative study confirms that two-step foresight offers the best trade-off between complexity and the robot's ability to accurately follow sequential step commands.
- **Modular Perception-Control Architecture:** A modular design separates perception from control, converting

high-level inputs into footstep targets tracked by the policy. This structure improves interpretability, and ensures physically valid commands.

II. RELATED WORKS

A. BIPED ROBOTS LOCOMOTION

Traditionally, classical control approaches for humanoid robot locomotion rely on a two-layer architecture. The high-level controller generates reference trajectories for the robot's joints or links using model-based techniques [1], [2], [3], [4]. These trajectories act as motion plans that guide the robot's movement. The low-level controller—such as Task-Space Inverse Dynamics (TSID) [5] or Inverse Dynamics Quadratic Programming (ID-QP) [6]—tracks these planned motions using full-body dynamics to ensure accurate execution. While these methods offer high precision and predictable performance, they depend heavily on accurate dynamic models and are often less adaptable to changing environments.

In recent years, model-free reinforcement learning has gained significant traction as a flexible alternative to classical control approaches [7]. Unlike classical methods, it does not require precise modeling of the robot's kinematics or dynamics. Instead, robots learn control strategies through trial and error, progressively improving their performance. This learning-based paradigm enhances adaptability, reduces reliance on accurate modeling, and increases robustness to uncertainties and hardware variations by training the robot with techniques such as dynamics randomization [8], [9]—making it potentially well-suited for real-world deployment.

While RL methods have shown great promise in the field of legged locomotion, many existing RL-based controllers do not explicitly account for footstep placement constraints [9], [10]. Instead, they often allow the controller to freely adjust foot positions to maintain balance. Although this flexibility simplifies the control task, it limits practicality in real-world scenarios—such as environments with unsteppable regions, potholes, or narrow footholds—where precise foot placement is critical. Most successful RL-based locomotion strategies focus on controlling base velocity [10] or learning rhythmic gaits [9], but they fall short in structured or unpredictable environments that demand high contact precision.

To enable locomotion in challenging terrains such as stairs, gaps, and uneven ground, prior studies have pursued two main directions. One line of work has investigated perception-free or “blind” locomotion, where the RL policy learns to handle obstacles purely through physical interaction. For example, Wang et al. proposed a concurrent teacher–student reinforcement learning architecture for legged robots [11], Siekmann et al. demonstrated blind walking with real-world stair ascent and descent [12], and Castillo et al. introduced a cascade reinforcement learning framework for 3D bipedal locomotion that achieves robust, perception-free walking over challenging terrains [13]. While effective in simulation and hardware, these approaches adapt only through

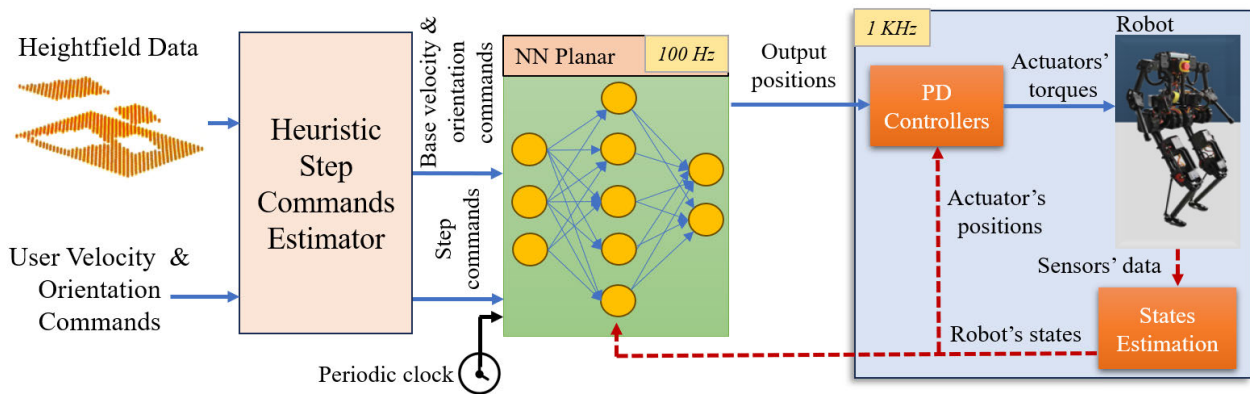


FIGURE 2. Overview of the structure and flow of the proposed learning-based framework. A heuristic step command estimator takes as input the user's velocity and orientation commands, along with height-field data when walking on non-flat terrain. The estimator determines the next two-step pose commands for each leg, as well as the average base orientation and velocity based on the upcoming steps, and passes them to the RL-based controller. The output of the RL policy is the desired joint positions. This modular structure improves robustness and interpretability, allowing the system to adapt effectively to both flat and uneven terrains. In the case of flat terrain, height-field data is not required. For uneven terrain, the height-field specifications (frontal and lateral ranges, as well as grid resolution) are provided in Section III-D-II.

leg–obstacle contact, as they lack visual input, external perception, or terrain models [7].

Complementary efforts have explored end-to-end reinforcement learning architectures that directly map perceptual inputs—such as heightmaps, depth images, or terrain reconstructions—to joint torques or positions [14], [15], [16], [17]. These methods enable visually guided locomotion but often lack interpretability and do not explicitly verify the safety or feasibility of foot placements. As a result, controllers may prioritize dynamic balance while selecting unsafe footholds, for instance, stepping on the edge of a stair and risking slippage or instability. This limitation is particularly critical for humanoid robots with flat feet, where secure, full-foot contact is essential for stability, as observed in the Cassie robot [14], where end-to-end vision policies frequently produced unsafe or inconsistent placements. Taken together, these results highlight the need for locomotion strategies that explicitly regulate footstep placement while leveraging perception to make terrain-aware decisions. Conceptually related, structured skill-based RL has been applied in humanoid agents for complex tasks, such as learning soccer behaviors using skill-set primitives in simulation [18], illustrating the benefits of task decomposition and modular policy design for improved control and interpretability.

In this work, we retain the adaptability benefits of RL while addressing the shortcomings of end-to-end designs. Specifically, we propose a structured RL-based controller that explicitly tracks commanded footstep poses, bridging the gap between high-level motion planning and low-level RL policy. This design improves interpretability, safety, and control precision in complex terrain settings.

B. FOOTSTEP PLANNING AND TERRAIN-AWARE LEARNING

Precise footstep planning is critical for navigating complex terrain, particularly when dealing with discrete footholds,

narrow surfaces, or gaps. In classical methods, footstep locations are typically generated using model-based approaches such as the Linear Inverted Pendulum Model (LIPM) and Capture Point concept [19], [20]. These approaches use simplified dynamics to compute feasible foot placements based on user-defined commands, such as walking speed and orientation. The resulting trajectories are tracked using low-level controllers like Whole-Body Control (WBC). While these methods produce physically consistent plans, their reliance on mathematical models and environment-specific assumptions makes it difficult to adapt to challenging terrain, stairs, or dynamically changing surfaces.

To overcome these limitations, RL has been adopted to introduce flexibility and adaptability to footstep planning. Some works employ RL at the navigation level, such as FootNet [21], where an RL-based planner generates footstep sequences to reach a distant target; these are then tracked by a downstream WBC. Others apply RL at the locomotion level, training policies to track predefined footstep sequences on flat terrain [22], [23], or to handle stairs using curriculum learning conditioned on future steps [24]. However, these approaches often assume fixed step lengths or struggle to generalize to terrain with varying step heights, randomly located footholds, or gaps.

Several hybrid and hierarchical RL frameworks have also been proposed. For example, Peng et al. [25] introduced the MACE framework, where multiple actor-critic pairs specialize in different terrain types, and the system selects between them at runtime. While effective, this architecture adds complexity and can be brittle in practice. Similarly, Deeploco [26] used foot placement as a guiding signal for RL to achieve path-following on flat terrain, but its reliance on motion priors and handcrafted step templates limited its adaptability.

The ALLSTEPS framework [27] introduced curriculum-driven learning for stepping stone navigation, where the policy is conditioned on the next two upcoming stone

positions. While effective for discrete foothold tracking, it does not support real-time footstep commands, limiting its flexibility for reactive or task-driven locomotion. We argue that enabling direct commandability of footstep plans is essential for terrain-aware and goal-oriented walking.

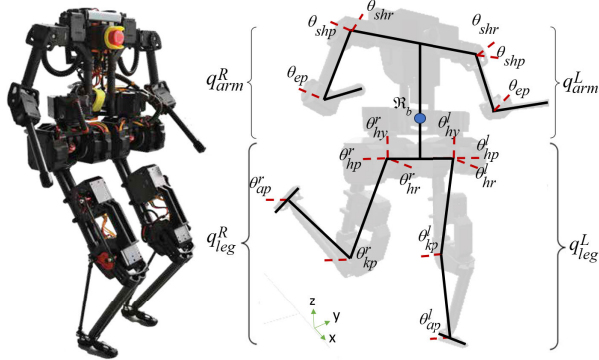


FIGURE 3. Bruce and its general morphology. The bipedal robot features 10 lower-limbs actuated joints—five per leg—and is designed for agile, dynamic locomotion.

Similarly, the method in [28] incorporates symmetry and periodicity into the reward function to improve footstep tracking but lacks the flexibility to adapt foot placements in response to hazards such as terrain gaps. Moreover, its footstep targets are generated using model-based ZMP planners and Fourier-series-based trajectory generators [29], which limit generalization. Model-based planners—such as those used in this work and in [30]—continue to rely on LIPM-based footstep generation. While these approaches offer interpretability, they often increase system complexity without delivering significant gains in performance, as we will demonstrate in a comparative study presented in this paper.

In contrast, our method presents a fully model-free, single-policy RL controller that tracks real-time footstep commands—including position, orientation, and height. This design enables reactive and terrain-aware walking without requiring motion priors, simplified dynamics, or handcrafted plans. By decoupling perception and control, our system maintains the benefits of modularity and safety, while leveraging the adaptability and robustness of data-driven learning.

To evaluate our method, we conduct simulations using BRUCE (Bipedal Robot Unit with Compliance Enhanced) [6], [32], a child-sized humanoid platform designed by RoMeLa in collaboration with Westwood Robotics. Bruce stands approximately 70 cm tall, weighs 4.8 kg, and is shown in Fig. 3. Each leg is designed with a 5-DoF kinematic chain consisting of a spherical hip joint (yaw, roll, and pitch), a knee pitch joint, and an ankle pitch joint. To reduce leg inertia, a cable-driven differential pulley system is applied to the hip joints, while a linkage mechanism is used at the ankle. These design choices minimize weight in the distal limbs and enhance compliance and backdrivability. In its nominal

configuration, Bruce is capable of walking at speeds up to approximately 0.8 m/s.

III. LEARNING FOOTSTEP-CONSTRAINED GAIT POLICIES

Our control policy directly incorporates footstep poses as control commands, replacing conventional base velocity or gait-pattern inputs. This policy accepts explicit footstep targets, including position, orientation, and height for upcoming steps (see Fig. 2). This design necessitates a specialized reward function that emphasizes accurate tracking of footstep commands, prioritizing precise foot placement over periodicity in motion.

A. STATE SPACE

The policy's state space includes both robot observations and user commands, forming a 60-dimensional vector $S \in \mathbb{R}^{60}$. Specifically, S consists of the base height relative to the current stance foot ($\mathbb{R}^1$), base linear and angular velocities ($\mathbb{R}^6$), base heading ($\mathbb{R}^1$), gravity projection in the base frame ($\mathbb{R}^3$), actuated joint positions and velocities ($\mathbb{R}^{20}$), commanded base velocity and orientation ($\mathbb{R}^3$), current foot positions and orientations relative to the base ($\mathbb{R}^8$), and a two-dimensional phase clock using sine and cosine functions ($\mathbb{R}^2$). Finally, the state representation includes the desired footstep poses for the next two steps of each foot—three steps for future planning and one already determined for the current stance foot. Each footstep pose is defined as:

$$U_j^i = [x, y, z, \theta] \quad \text{for } i \in \{l, r\}, j \in \{1, 2\}, \quad (1)$$

resulting in a 16-dimensional input ($\mathbb{R}^{16}$).

In this work, all observation data are obtained directly from the simulation environment, but these signals are readily available on modern humanoid robotic platforms, including Bruce. In a real robot, this information can be obtained using joint position and velocity sensors, together with an IMU that provides base orientation and linear/angular velocities. Foot positions in the robot's local frame and base height can then be estimated from these sensors using forward kinematics.

B. ACTION SPACE

The action space $A \in \mathbb{R}^{10}$ corresponds to PD setpoints for each motor. Actions are generated at 100 Hz and executed through a joint-level PD controller operating at 1 kHz with fixed gains, following the setup in [30]. While 100 Hz provides additional margin for dynamic behaviors and aligns with prior work, the method remains applicable at 50 Hz, as is common in many existing systems, since the low-level controller and estimation modules run at higher rates.

The arms of Bruce are assumed to be fixed in this study to reduce the dimensionality of the control problem and to focus the training on legged locomotion. While free arms could indeed contribute to improved balance and agility [31], incorporating them would increase the action space and the complexity of training. We consider arm motion as an extension for future work, where the additional degrees of

freedom can be exploited for enhanced disturbance rejection and whole-body coordination.

C. REWARD FUNCTIONS

1) CONTACT SCHEDULE REWARD

To encourage accurate stance foot tracking, we penalize deviations from the commanded foot pose $p_{\text{cmd}}(T) = [x, y, z, \theta]$, which is defined on a step-to-step basis. The reward is defined following the method described in [30] as:

$$r_{\text{cs}} = w_{\text{cs}}(I_{r,\text{contact}} - I_{l,\text{contact}}) \phi_{\text{contact}} \exp\left(-\frac{\|p^{\text{st}} - p_{\text{cmd}}^{\text{st}}\|_2}{\sigma}\right), \quad (2)$$

where p^{st} is the current stance foot position; w_{cs} is the reward weight; and ϕ_{contact} is a phase-based function oscillating between -1 and 1 :

$$\phi_{\text{contact}} = \frac{\sin(2\pi\phi)}{\sqrt{\sin^2(2\pi\phi) + 0.04}}, \quad \phi = \frac{t'}{2T}, \quad (3)$$

with t' as the elapsed time since the start of the current right-foot step, which resets to 0 at the start of each two-step cycle. $I_{i,\text{contact}}$ is a binary indicator that determines whether the specified foot i is currently in contact with the ground. σ in the reward function is the shaping parameter for the exponential function.

2) SWING-FOOT TRAJECTORY TRACKING

To encourage the swing foot to follow a smooth and natural trajectory during its swing phase, we define a reference path $p_{\text{ref}}(\phi_{\text{sw}})$, parameterized by $\phi_{\text{sw}} = t''/T \in [0, 1]$. Here, t'' denotes the elapsed time since the beginning of the current swing phase.

Let $p_{\text{start}} = (x_s, y_s, z_s)$ and $p_{\text{end}} = (x_e, y_e, z_e)$ represent swing start and end positions. We first interpolate the horizontal (x, y) position linearly by:

$$(x(\phi_{\text{sw}}), y(\phi_{\text{sw}})) = (1 - \phi_{\text{sw}})(x_s, y_s) + \phi_{\text{sw}}(x_e, y_e). \quad (4)$$

To compute the height $z(\phi_{\text{sw}})$, we define a parabolic arc over the step. The midpoint (in x - y) and radius r are given by:

$$c = \frac{1}{2}((x_s, y_s) + (x_e, y_e)), \quad (5)$$

$$r = \frac{1}{2}\|(x_e, y_e) - (x_s, y_s)\|. \quad (6)$$

We define the apex height as:

$$h_{\text{apex}} = \max(z_s, z_e) + \alpha \cdot r, \quad (7)$$

where α is a tunable hyperparameter representing the apex height percentage. Then, the vertical displacement due to the parabola is given by:

$$\Delta z(\phi_{\text{sw}}) = -\frac{\|(x(\phi_{\text{sw}}), y(\phi_{\text{sw}})) - c\|^2}{(r/\sqrt{h_{\text{apex}}})^2} + h_{\text{apex}}, \quad (8)$$

The final height blends the start and end foot heights with the parabolic displacement:

$$z(\phi_{\text{sw}}) = (1 - \phi_{\text{sw}})z_s + \phi_{\text{sw}}z_e + \Delta z(\phi_{\text{sw}}). \quad (9)$$

This trajectory is used to compute a swing foot tracking reward by penalizing deviation from $p_{\text{ref}}(\phi_{\text{sw}})$ during the swing phase:

$$r_{\text{sft}} = w_{\text{sft}} \exp\left(-\frac{\|p^{\text{sw}} - p_{\text{ref}}^{\text{sw}}\|_2}{\sigma}\right), \quad (10)$$

where p^{sw} is the actual swing foot position; w_{sft} is the reward weight.

3) COMMAND FOLLOWING REWARDS

To encourage the robot to react to sudden environmental changes, we define additional reward terms that track commands for body orientation and base velocity, in addition to the footstep commands. These high-level commands are computed by averaging the corresponding values over a future horizon of steps. This multi-step formulation prevents myopic behavior; for instance, the robot does not simply pitch its body forward for the next step but also considers the requirements of subsequent steps. The base forward velocity command, for example, is calculated as follows:

$$v_{\text{avg}}^{\text{base}} = \text{mean}\left(\frac{L_s^j}{T}\right), \quad j \in \{1, 2, 3\}, \quad (11)$$

Then the reward is calculated as:

$$r_{\text{vel}} = w_{\text{vel}} \exp\left(-\frac{\left\|\frac{v - v_{\text{avg}}^{\text{base}}}{1 + |v|}\right\|^2}{\sigma}\right), \quad (12)$$

where L_s^j is the step length for the j next step; v is the frontal velocity of the robot written in its base frame. In the same way, the reward function to follow the body orientation is defined by:

$$r_{\text{orient}} = w_{\text{orient}} \exp\left(-\frac{|\theta_{\text{base}}|}{\sigma}\right). \quad (13)$$

θ_{base} is the current yaw orientation of the base in the world frame. w_{vel} and w_{orient} are the reward weights.

4) REGULARIZATION REWARDS

To mitigate any undesirable motions, a set of regularization rewards is imposed to penalize excessive joint torque, velocities, unnecessary angular motion, policy termination due to falls, etc. Those functions are presented as in [30] with some modifications to fit our problem and with regulating the weights to fit our robotic model.

To ensure that the actions produced by the RL policy remain within the robot's physical capabilities, the reward function is designed to account for the 3-DoF kinematic chain of the robot's legs and the four-bar linkage mechanism at the knee joint. This is achieved by incorporating kinematic

constraints derived from the analytical models of these mechanisms as described in [6].

The weights of all previous reward functions are presented in Table 1. The reward shaping parameter σ for the exponential function is set to 0.25 during training. The reward weights were chosen empirically to achieve stable locomotion while prioritizing accurate footstep and orientation tracking. We iteratively adjusted them to balance stability, smoothness, and compliance, selecting the set yielding the best overall performance.

TABLE 1. Reward Functions Weights.

Reward	Weight
Joint torques	1×10^{-4}
Joints torque limits	0.01
Joint velocity	1×10^{-3}
Joints limits	10
Base height	3
Base heading	3
Contact schedule	9
Base tilting	1
Action smoothness 1	0.1
Action smoothness 2	0.005
Hip joints regularization	1
Base roll-pitch velocity	0.01
Foot heading	3
Velocity tracking	2
Swing reference trajectory tracking	9
Termination	100

D. POLICY COMMAND GENERATION

To train the robot to follow random step commands, we require a mechanism for generating feasible footstep targets that respect the robot's physical constraints. To this end, we design a velocity-constrained sampling procedure that sequentially generates footstep poses. This sampling process is invoked at the beginning of each walking phase to produce valid step commands.

Furthermore, the training is structured into two distinct stages. In the first stage, the robot learns to follow randomly generated footstep targets on flat terrain. In the second stage, training continues on more challenging terrain featuring obstacles with randomly varying heights, enabling the robot to generalize its behavior to more complex environments.

1) FLAT TERRAIN STEP COMMANDS

The system maintains a four-step prediction horizon. At each gait cycle, a moving window is applied to ensure smooth transitions: previously planned steps are shifted forward in time (i.e., the third command becomes the second, the second becomes the first), and a new step command is generated to update the final position in the sequence. Step timing is managed through phase-based detection rather than direct foot contact sensing, which enhances robustness to sensor noise and facilitates potential real-world deployment.

Each gait cycle begins with the computation of a velocity vector that forms the basis for step placement. The forward velocity component v_x is randomly perturbed within a bounded range $[-0.5, 1.0]$ m/s, while the lateral

velocity component v_y is sampled from $[-0.3, 0.3]$ m/s. A probabilistic retention mechanism with probability $p_{\text{keep}}^{\text{vel}} = 0.8$ governs whether the previous velocity command is retained or replaced with a newly sampled value. This probabilistic strategy ensures diversity in generated steps while helping the robot learn to follow commanded velocities more effectively. The selected velocity ranges are consistent with the default controller of BRUCE, which supports stable walking up to approximately 0.8 m/s. By training within and slightly beyond this range, we ensure that the learned policy remains both realistic and robust.

Foot orientation commands θ are generated in a similar fashion, sampled from the range $[-1, 1]$ rad and retained with a probability of $p_{\text{keep}}^{\theta} = 0.9$.

Foot placements in the world frame are computed using the transformation:

$$\begin{aligned} x_n &= x_{n-1} + v_x T \cos \theta + 2 v_y T \sin \theta, \\ y_n &= y_{n-1} + v_x T \sin \theta - 2 v_y T \cos \theta, \end{aligned} \quad (14)$$

where (x_n, y_n) represents the n -th footstep position, T is the step duration (0.3 s for our robot), and θ is the heading angle. To avoid foot collisions, we enforce a minimum step width of $d = 0.12$ m: after computing the nominal step, we adjust it whenever the inter-foot distance falls below this threshold. The lateral velocity terms are scaled by a factor of 2 because, unlike forward motion, the swing foot must remain on its respective side of the sagittal plane (i.e., the left foot on the left side and the right foot on the right side).

Finally, the footstep position is expressed relative to the robot's base position. This formulation ensures velocity-proportional step lengths, a constant minimum spacing for balance, and directional consistency in the generated footsteps.

2) UNEVEN TERRAIN STEP COMMANDS

The process of generating step commands in random discrete steps terrain follows a similar strategy to that used on flat ground. However, at the beginning of each gait cycle, the system additionally computes the height of each planned footstep. To avoid placing the foot near terrain edges, a positional offset is applied when necessary. This procedure relies on a measured height field, which provides an estimate of the ground elevation in front of the robot. For the Bruce robot, it is sufficient to sample this field within a frontal range of $[-0.1, 0.8]$ meters and a lateral range of $[-0.3, 0.3]$ meters relative to the robot base. The measurement region is discretized into a 33×33 grid, offering reliable spatial resolution for footstep planning. In a real robot, this height-field data can be obtained using LiDAR, depth cameras, or stereo vision. Bruce in its current form does not have any height-field sensors; however, the hardware could be updated to include them, as in [14]. Furthermore, this method can be applied to any humanoid robot equipped with such sensors.

To ensure safe and stable foot placement, the system first evaluates local terrain variability at each step location. This is achieved by identifying the closest point from a precomputed variance map and assessing how uneven the surrounding area is. If the step is found to fall on an edge, it is shifted to a nearby flatter region by selecting the closest point with sufficiently low variance. Once the position is adjusted, the step height is estimated by querying the nearest terrain height point. The final footstep height is then expressed relative to the height of the robot base from the ground, supporting smooth execution and improved adaptability to challenging terrain. The overall pseudocode of the algorithm is displayed in Algorithm 1, where E denotes the number of environments used during training, and X and Y represent the grid resolution (33×33).

Algorithm 1 Terrain-Aware Step Adjustment

Input: measured-heights: Raw terrain height samples

threshold: Variance threshold for flat terrain

step-commands: Desired step locations

stance-height: Current stance foot height

Output: Adjusted step positions and height commands

1. Terrain Variance Estimation:

Reshape input:

$\mathcal{H} \leftarrow \text{measured-heights} \in \mathbb{R}^{E \times 1 \times X \times Y}$

Define smoothing kernel: $\mathbf{K} \leftarrow \frac{1}{15} \mathbf{1}^{5 \times 3}$

Compute average height: $\tilde{\mathcal{H}} \leftarrow \text{conv2d}(\mathcal{H}, \mathbf{K})$

Compute squared difference: $\Delta^2 \leftarrow (\mathcal{H} - \tilde{\mathcal{H}})^2$

Compute variance map: $\mathcal{V} \leftarrow |\text{conv2d}(\Delta^2, \mathbf{K})|$

2. Step Position Adjustment:

foreach step command $\mathbf{p} \in \text{step-commands}$ **do**

Find nearest heightfield point:

$\mathbf{v} \leftarrow \arg \min_i \|\mathbf{v}_i^{xy} - \mathbf{p}^{xy}\|$

if $\mathbf{v}^z \geq \text{threshold}$ **then**

Find closest flat point:

$\mathbf{f} \leftarrow \text{first}(\mathbf{v}_j \mid \mathcal{V}_j < \text{threshold})$

Update XY position: $\mathbf{p}^{xy} \leftarrow \mathbf{f}^{xy}$

else

└ Fallback to nearest point: $\mathbf{f} \leftarrow \mathbf{v}$

Set step height command:

height-cmd $\leftarrow \mathbf{f}^z - \text{stance-height}$

E. DOMAIN RANDOMIZATION

Although our experiments are conducted in simulation, we employ domain and dynamics randomization during training to improve robustness, stabilize learning, and reduce overfitting to simulator-specific parameters [8], [33]. This strategy is consistent with established findings that randomized training reduces the simulation-to-real gap by encouraging policies that tolerate modeling errors, sensing noise, and actuation uncertainties [8], [34].

In practice, we randomize terrain heightmaps as well as key physical and contact parameters (e.g., mass, friction).

We also inject noise into proprioceptive measurements (e.g., joint encoders and IMU) to model sensing imperfections. For structured terrains such as stairs, steps, and gaps, additional noise is applied to the heightmap measurements to emulate perception errors [14]. During training, the robot is subjected to external disturbances applied as instantaneous linear velocity perturbations in random directions within the XY plane. These disturbances are applied once every 10 seconds of simulated time. This approach, commonly used in reinforcement learning for legged robots, simulates the effect of sudden pushes without applying direct forces.

The full set of parameter ranges is summarized in Table 2. These values were chosen to cover realistic variations in robot dynamics and sensor noise while ensuring stable and efficient training in simulation.

TABLE 2. Domain randomization parameters applied during training. Noise values are standard deviations where applicable.

Category	Parameter	Range / Std	Unit
Simulation Model	Friction coefficient	[0.5, 1.4]	–
	Added base mass	[-0.5, 0.5]	%
	Base position offset	[-0.01, 0.01]	m
Heightmap Noise	Gaussian noise in XY	0.02	m
	Gaussian noise in Z	0.02	m
External Disturbances	Push interval	10	s
	Max push velocity (XY)	0.5	m/s
Sensor Noise (Std)	Base height	0.02	m
	Base linear velocity	0.05	m/s
	Base heading	0.01	rad
	Base angular velocity	0.05	rad/s
	Projected gravity	0.05	–
	Foot state	0.01	m
	DOF position	0.05	rad
	DOF velocity	0.5	rad/s

F. LEARNING ALGORITHM

Proximal Policy Optimization (PPO) [35] is a widely used on-policy reinforcement learning algorithm, known for its stability and efficiency in high-dimensional control tasks. It strikes a balance between exploration and exploitation by ensuring that policy updates are neither too small (which slows down learning) nor too large (which risks performance collapse). PPO is particularly well-suited for environments like Isaac Gym [36], where multiple agents can be trained in parallel.

The central idea in PPO is to maximize a clipped surrogate objective function, defined as:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (15)$$

Here, $r_t(\theta)$ is the probability ratio between the new policy π_θ and the old policy $\pi_{\theta_{\text{old}}}$:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \quad (16)$$

The term $\hat{A}_t$ represents the estimated advantage function at timestep t , which indicates how much better or worse

TABLE 3. Training Hyperparameters.

Hyperparameter	Value
Number of agents	8192
Number of steps in iteration	24
Entropy coefficient	0.01
Value function coefficient	1.0
Clipping	0.2
Gamma	0.99
Lambda	0.95
KL target	0.01

an action performed compared to the expected baseline. To estimate $\hat{A}_t$, we use Generalized Advantage Estimation (GAE):

$$\hat{A}_t = \sum_{l=0}^{T-1} (\gamma \lambda)^l \delta_{t+l} = \delta_t + (\gamma \lambda) \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + \dots + (\gamma \lambda)^{T-1} \delta_{t+T-1} \quad (17)$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (18)$$

Here, T denotes the number of time steps used for advantage estimation, corresponding to the trajectory horizon length used during training (e.g., $T = 24$ in our setup).

The clipping operation in the loss function ensures that updates do not move the policy too far from the current one, controlled by the hyperparameter ϵ (commonly set between 0.1 and 0.3). This technique prevents destructive policy updates and encourages smoother convergence.

In our setup, both actor and critic networks are implemented as LSTMs with three layers and 256 hidden units each. Compared to feedforward policies, LSTMs provide internal memory, enabling the policy to exploit temporal dependencies, reject disturbances more effectively, and encode latent dynamics for improved robustness. Such benefits of memory-based control in biped locomotion have also been demonstrated in prior work [33], where RNNs were shown to outperform memoryless policies when dynamics randomization was used to prevent overfitting. The training benefits from Isaac Gym's GPU-accelerated parallel simulations, allowing us to train thousands of agents simultaneously for rapid policy refinement. Training hyperparameters are detailed in Table 3.

IV. SIMULATION RESULTS

In this section, we present the simulation results of applying our controller. We first trained the policy on flat terrain without any height information (the step height command is set to zero). Subsequently, the same policy was trained on terrain with randomly placed discrete steps. This training setup was chosen to ensure that the robot maintains its ability to walk effectively on flat terrain while simultaneously gaining experience in handling variations in ground height. The reader is referred to the accompanying submission video for demonstration results.

A. IMPORTANCE OF FORESIGHT IN STEP PLANNING

When the robot walks on flat terrain with fixed step lengths and no orientation changes, it does not face significant

challenges in following step commands, even with moderately frequent changes in velocity. In fact, under this scenario, step planning becomes largely unnecessary — the robot can be trained simply to follow velocity commands to achieve the locomotion task.

However, the benefits of step-foresight planning become evident when step positions or orientations must change more frequently. This scenario typically arises in dynamic environments or terrains with randomly placed obstacles, where the robot must adjust its steps continuously to maintain stable locomotion — for example, when traversing a gapped terrain. In such situations, a key question arises: to what extent should we plan the robot's steps, and what is the optimal number of foresight steps needed to ensure stable locomotion?

Here, we distinguish between strategic planning, which guides the robot through the environment toward a target location, and step planning, which ensures stable locomotion by selecting feasible footstep sequences.

To investigate the impact of planning horizon on performance, we designed three policies based on the same network architecture described in Section III, varying only the number of foresight-planned steps: one-step foresight per leg, two-step foresight (our proposed method), and three-step foresight.

Evaluation of these methods on flat terrain with moderate velocity changes shows that all three perform similarly, as expected — the robot is able to follow the commanded footsteps and thus achieve the desired walking velocity.

To assess performance in dynamically changing environments, we simulated the robot walking for five minutes on flat terrain while receiving randomly generated footstep pose commands. The position and orientation targets were sampled from the same ranges used during training. A step was considered successful if the robot placed its foot within 3 cm of the target position and within 10 degrees of the desired orientation. The overall success rate was calculated as the percentage of successful steps during the simulation.

Fig. 4 presents the success rates for following footstep position and heading commands for the three methods. From the figure, we observe that for step position tracking, the two- and three-step planning methods yield similar results and outperform the one-step method. This is because having foresight about upcoming step changes allows the robot to adjust its body velocity in advance, thereby improving step tracking. However, we also find that two-step foresight is sufficient for this task, and adding more steps does not offer additional benefits.

Regarding step orientation tracking, we similarly find that two-step foresight is sufficient. Interestingly, the three-step method performs worse in this case. This may be due to the way the robot averages its body heading over a longer horizon, which could prevent it from aligning accurately with immediate orientation targets.

In conclusion, our results suggest that a two-step foresight strategy is sufficient to achieve the locomotion goals in our

study. It balances responsiveness and foresight, ensuring stable and accurate footstep tracking in dynamic environments.

B. WALKING ON FLAT TERRAIN

In this section, we conduct several tests to evaluate the performance of our method during flat-terrain walking and compare it with the model-based approach presented in [30].

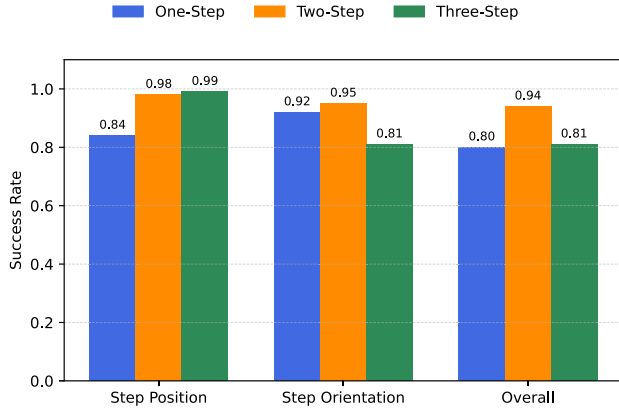


FIGURE 4. Success rate of following footstep poses commands. The results shows that the walking with two-step a head planning gives us the best overall results.

TABLE 4. Statistics of footstep tracking errors for the proposed method and the LIPM-based method.

Metric	Proposed Method		LIPM-based Method	
	Mean	Std.	Mean	Std.
Step Location Offset (m)	0.004	0.004	0.007	0.005
Step Heading Offset (rad)	0.065	0.055	0.089	0.054

1) TRACKING DESIRED STEP LOCATION

Fig. 5 illustrates the robot walking effectively on flat terrain while following randomly generated step positions and orientation commands.

Fig. 6 shows the generated foot trajectories for the left and right legs. The robot demonstrates reliable tracking of the planned step locations, resulting in accurate and stable foot placements at the end of each swing phase.

Table 4 shows the mean absolute error and standard deviation of the footstep tracking offsets. These measurements were calculated over a five-minute simulation of the robot walking on flat terrain while receiving randomly generated footstep pose commands within the same ranges used during training.

The results indicate that the proposed method achieves accurate footstep command tracking. Compared to the model-based approach, our method attains slightly better performance. The generally good results of both methods stem from the low-level controller—in our case, the reinforcement learning policy—being primarily responsible for step tracking. Thus, the choice of high-level planner has a limited impact on accuracy, provided the commands are reasonable and the robot is well trained. The main

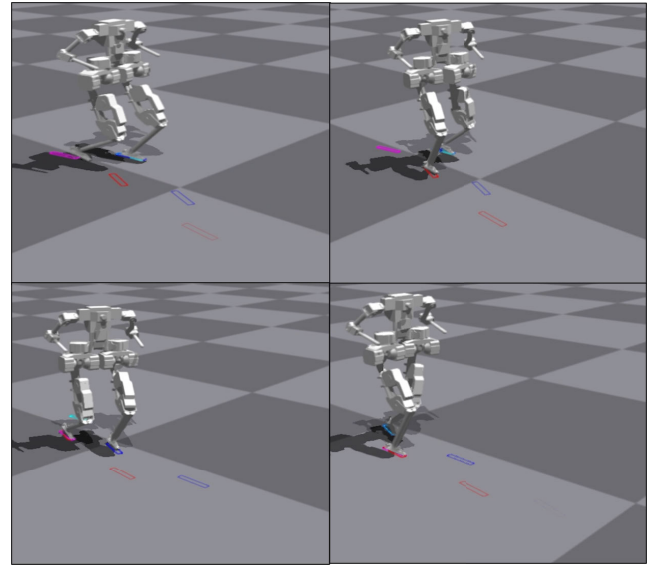


FIGURE 5. Evaluation of the proposed approach for walking on flat terrain. The figure shows that the robot closely follows the commanded step positions and orientations, demonstrating accurate foot placement during walking. Red: commanded right footstep; Pink: actual right footstep; Blue: commanded left footstep; Cyan: actual left footstep.

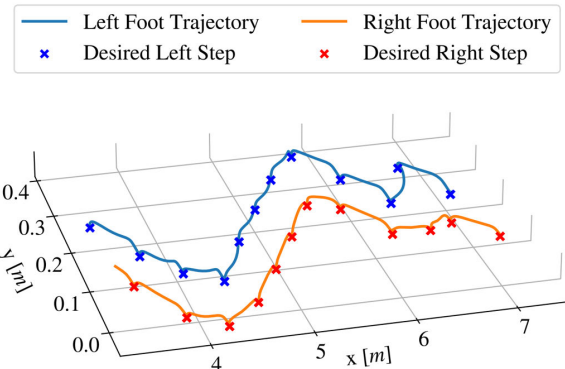


FIGURE 6. Tracking the desired step location along with the resulting foot trajectory at randomly varied step positions and orientations. The figure demonstrates precise and stable foot placements achieved at the end of each swing phase.

advantage of our approach lies in the foresight of two-step lookahead planning, which enables more dynamic adaptation to changing commands, as shown in Section IV-A.

2) VELOCITY TRACKING PERFORMANCE

Our control policy tracks discrete footstep position and orientation commands, which are computed based on the velocity command provided at the beginning of each step phase. The planner does not update footstep targets in response to velocity changes during the phase. Nonetheless, as shown in Fig. 7, the robot effectively follows the commanded velocities set at the start of each phase in both longitudinal and lateral directions.

The coordination between footstep planning and base motion, along with the employed strategy for generating velocity commands, enables the robot to maintain smooth and

stable walking dynamics. Furthermore, the comparison with the model-based method demonstrates that incorporating a dynamic model to generate step commands does not provide additional benefit, which further validates the effectiveness of our approach.

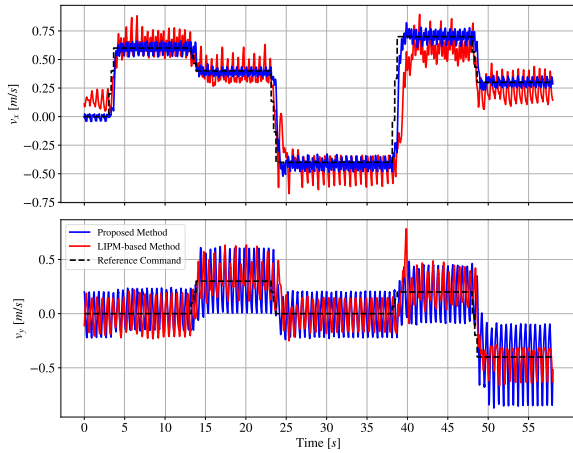


FIGURE 7. Velocity tracking performance on flat terrain. The results demonstrate that although the proposed approach relies on tracking footstep position commands, it successfully achieves accurate velocity tracking. The controller effectively tracks velocity commands in the horizontal plane, including forward/backward (v_x), left/right (v_y).

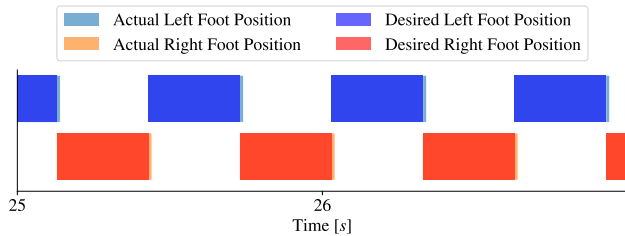


FIGURE 8. Evaluation of the proposed approach for walking on flat terrain. A contact schedule reward was used to encourage a step duration T of 0.3 s. The discrepancy between the actual and target foot contact timings remained below 0.005 s.

3) STEP DURATION TRACKING

As shown in Fig. 8, the learned policy consistently achieves foot contact durations close to the target value of 0.3 s, with contact schedule errors remaining below 0.005 s. Such precise timing is advantageous for integrating phase-based state estimators in real-world deployments [30]. The LIPM-based method exhibits comparable performance for this metric; therefore, its results are not included here.

4) ENERGY EFFICIENCY

To evaluate the energy efficiency of the proposed controller, we use the cost of transportation (CoT) metric [37]. The total mechanical work of the actuation system is defined as

$$E_m = \int_0^T \sum_{i=0}^N |\tau_i(t) \omega_i(t)| dt, \quad (19)$$

while the positive mechanical work, which accounts only for energy generation, is given by

$$E_{m+} = \int_0^T \sum_{i=0}^N \tau_i(t) \omega_i(t) dt, \quad \text{if } \tau_i(t) \omega_i(t) > 0. \quad (20)$$

Here, t denotes the time step, T is the walking duration, m , N is the number of actuators, and τ_i and ω_i represent the torque and angular velocity of motor i , respectively.

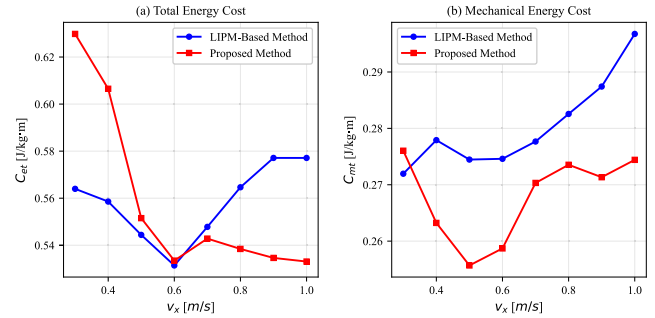


FIGURE 9. Comparison of energy costs between the LIPM-based method and the proposed approach: (a) Total energy cost (C_{et}), (b) Mechanical energy cost (C_{mt}). The proposed method shows improved energy efficiency across most speed ranges.

To allow a fair comparison across different robotic systems, the energy values are normalized by the robot's weight and traveled distance, leading to the following definitions for the energetic cost of transport C_{et} and the mechanical cost of transport C_{mt} :

$$C_{et} = \frac{E_m}{mgD}, \quad C_{mt} = \frac{E_{m+}}{mgD}, \quad (21)$$

where m is the total mass of the robot, g is the gravitational acceleration, and D is the traveled distance.

For the evaluation, the robot was commanded to walk on a flat surface in the frontal direction, with velocities ranging from 0.3 m/s to 1.0 m/s. For each velocity command, incremented in steps of 0.1 m/s, the robot executed 100 steps, and the average energy consumption was computed. The results are illustrated in Fig. 9, while the overall performance within the selected range is summarized in Table 5. The frontal direction was chosen for evaluation, as it provides a clear indication of the robot's performance and reflects the primary locomotion direction in most real-world scenarios.

Compared to the model-based method, the proposed controller achieves higher mechanical energy efficiency across most speeds, generating forward motion with less propulsive effort by leveraging the robot's passive dynamics (e.g., natural link swing and gravity). However, at lower speeds, the total cost of transport (C_{et}) is higher for the proposed controller, suggesting increased negative work or dissipative losses such as braking torques and postural maintenance. Interestingly, this trend is qualitatively similar to that observed in human walking [38], where the energetic cost is elevated at slow speeds due to less effective use of passive dynamics and greater reliance on stabilization torques.

TABLE 5. Mean and Standard Deviation of Energy Efficiency Metrics.

Metric	LIPM-Based Method	Proposed Method
C_{et} Mean (J/kg·m)	0.5640	0.5331
C_{et} Std. (J/kg·m)	0.0181	0.0386
C_{mt} Mean (J/kg·m)	0.2805	0.2683
C_{mt} Std. (J/kg·m)	0.0084	0.0081

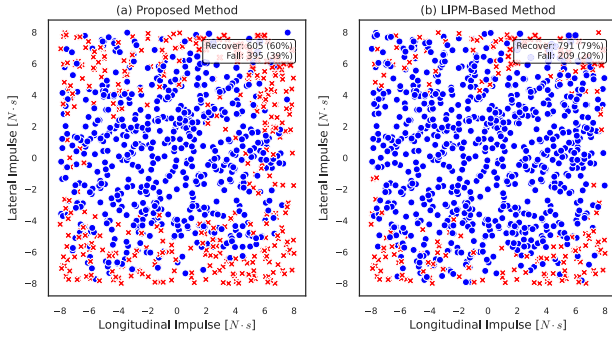


FIGURE 10. Robot's response to external pushes at zero velocity ($I_{\max} = 8 \text{ N}\cdot\text{s}$). Each subplot shows the distribution of points where the robot either recovered or fell, out of 1000 trials. The numbers of successful recoveries and falls are indicated within each subplot. The LIPM-based method achieved a 79% success rate, a 32% relative improvement over our method's 60% success rate. Symbols: 'o' = recovery, 'x' = fall.

5) ROBUSTNESS

The robustness of the proposed method was evaluated by applying external linear pushes to the robot's base and recording its response (fall or recovery). Two test scenarios were considered. In the first, the robot was instructed to maintain zero velocity throughout the trials. In the second, the robot was given random velocity commands in the x - and y - directions within its operational ranges, namely

$$v_x \in [-0.5, 1.0] \text{ m/s}, \quad v_y \in [-0.3, 0.3] \text{ m/s}.$$

Each experiment consisted of 1000 trials, with maximum impulse values of $I_{\max} = 8 \text{ N}\cdot\text{s}$ in the x - and y -directions.

Fig. 10 shows the distribution of coordinates where the robot either fell or successfully recovered while in stance. The LIPM-based method achieved a 79% success rate, a 32% relative improvement over our method's 60% success rate.

The recovery success rates in the second scenario were 59.1% for the proposed method and 73.9% for the LIPM-based method.

The results indicate that our method achieves robust performance, though the LIPM-based method demonstrates a measurable advantage in robustness. We attribute this to the fact that LIPM explicitly determines the next footstep command based on the actual velocity of the robot's base, effectively incorporating a feedback mechanism that enhances stability. In contrast, our approach relies solely on the experience gained during training. Nevertheless, the results demonstrate that our method also achieves reliable robustness. We note that its stability could be further

improved by training with stronger disturbances. However, in this study, we preferred that all methods be trained under identical conditions with random dynamics and disturbances to ensure a fair comparison.

6) DISCUSSION

The results presented in this section demonstrate that our method performs well across all evaluated metrics on flat terrain. When compared to the model-based approach, explicitly incorporating robot dynamics does not yield a significant improvement in overall performance, except in disturbance rejection, where the feedback mechanism of the LIPM-based method provides an advantage. However, the simplicity of our approach makes it more amenable to generalization, enabling extensions to more complex scenarios such as stair climbing and locomotion over uneven terrain with steps. This highlights the effectiveness and practicality of the proposed method.

We acknowledge that performance may vary depending on the specific policy obtained during training, which in turn can differ due to variations in initial conditions, network weights, or random seeds. Nevertheless, the reported results provide clear evidence that our method is effective and that reliance on a model-based approach is not essential for achieving robust footstep tracking.

C. WALKING ON TERRAIN WITH GAPS

To demonstrate the effectiveness of the two-step-ahead footstep planning strategy, we evaluate the robot's performance on terrain with gaps using the same policy trained on flat terrain. Terrain perception is based on a height field, consistent with the training setup for navigating discrete steps. When a gap is detected at the location of the next planned footstep, the robot attempts to shift the step to the nearest feasible flat surface. If no suitable foothold is found within the robot's kinematic limits, the robot halts safely at the edge of the gap. A similar approach was adopted in [30] to enable gap traversal in legged robots.

Fig. 11 shows the robot successfully navigating over terrain with gaps while following the generated step commands at a walking speed of 0.6 m/s. This demonstrates the effectiveness of our method in maintaining stability while traversing discontinuous surfaces.

The benefits of the two-step-ahead strategy become especially evident in these scenarios. If the robot detects that the second future step lies within a gap that it cannot cross, it proactively adjusts the position of the preceding step to reduce the commanded walking velocity. This allows the robot to gradually decelerate and come to a controlled stop before reaching the gap. To highlight this advantage, we compared it with a one-step-ahead strategy. In that case, the robot was unable to react in time due to its forward momentum and ultimately fell into the gap. This comparison underscores the critical role of long-horizon footstep planning in ensuring safe and adaptive locomotion on challenging terrain.

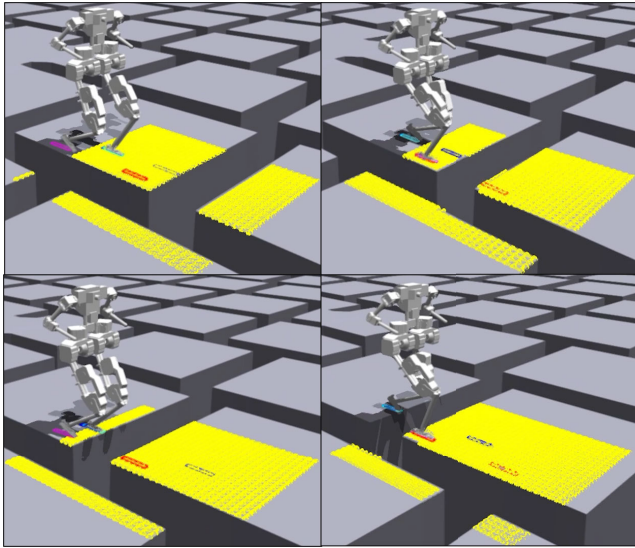


FIGURE 11. Evaluation of the proposed approach for walking on terrain with gaps. The figure demonstrates that the robot successfully crosses the gaps when their widths are within the robot's maximum feasible step length. Yellow points represent the height map of the terrain. Red: commanded right footstep; Pink: actual right footstep; Blue: commanded left footstep; Cyan: actual left footstep.

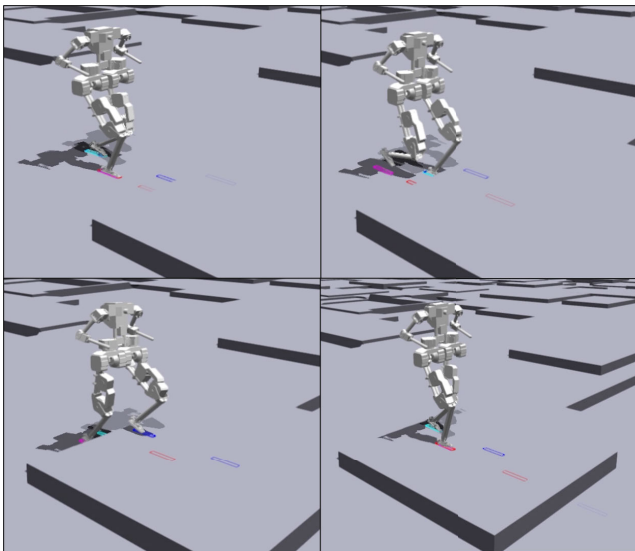


FIGURE 12. Evaluation of the proposed approach for walking on terrain with random discrete steps. The robot was able to ascend steps with heights of up to approximately 33% of its leg length, demonstrating robust locomotion over uneven terrain. Red: commanded right footstep; Pink: actual right footstep; Blue: commanded left footstep; Cyan: actual left footstep.

D. WALKING ON RANDOM DISCRETE STEP TERRAIN

To train the robot, we generate random discrete steps with varying heights in the range of $[0.05, 0.1]$ m. Given that the leg length from the hip yaw joint to the foot is approximately 30 cm, the maximum step height represents about 33% of the leg length. This setup provides a challenging terrain for the robot and helps improve its adaptability and robustness during training.

The robot was able to successfully traverse the terrain with random discrete steps. Throughout the process, its

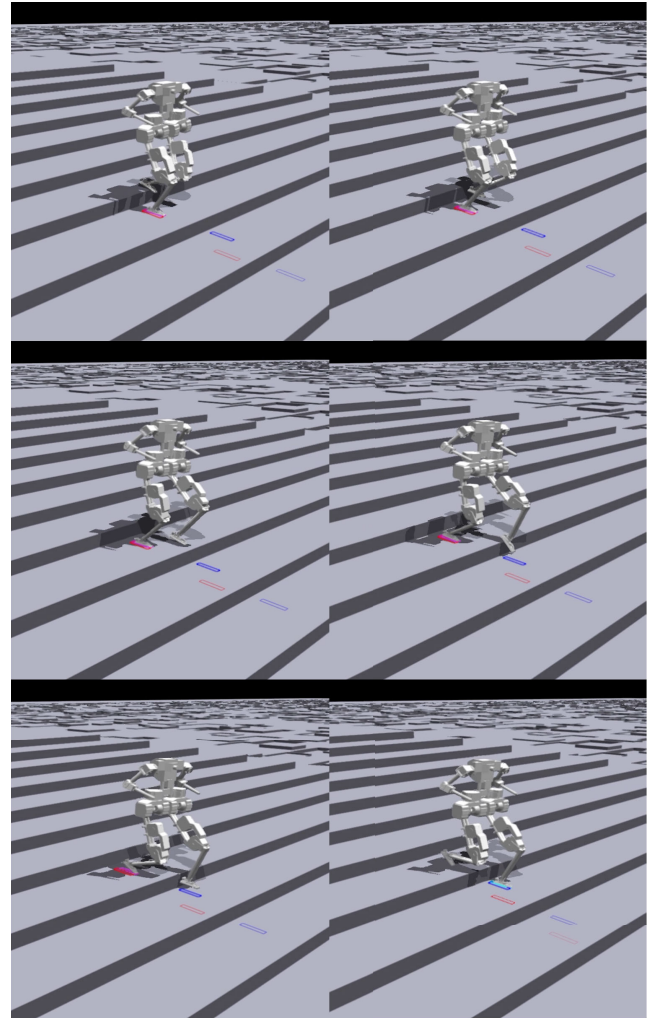


FIGURE 13. Evaluation of the proposed approach for descending stairs. The robot successfully navigates downward steps by accurately placing its feet and adjusting its posture to maintain balance. This demonstrates the controller's capability to handle negative height variations and maintain stable locomotion on descending terrain.

performance remained stable and reliable. The robot's ability to handle the varied step positions was evident. A snapshot of the robot passing over a step is shown in Fig. 12.

We also evaluated the robot's ability to handle obstacles beyond the height range it was trained on. Specifically, we tested its performance on obstacles reaching up to 50% of its leg length—significantly higher than the 33% used during training. The robot was able to successfully adjust its foot height to clear these taller obstacles, demonstrating the robustness and generalization capability of the proposed controller. This result also confirms that the robot effectively relies on height readings to determine appropriate foot lift, rather than memorizing specific training patterns.

Using the same trained policy, we also evaluated the robot's ability to ascend and descend stairs with a step height of 8 cm—scaled to match the standard stair height relative to a human leg. Fig. 13 shows snapshots of the robot successfully walking up and down the stairs.

V. CONCLUSION AND FUTURE WORK

In this work, we presented a learning-based step-following controller capable of tracking footstep positions and orientations while adaptively adjusting step height using terrain information. The approach demonstrated robust performance in simulation, successfully traversing complex terrains with gaps and obstacles up to 50% of the robot's leg length and tracking speeds up to 0.8 m/s. By decoupling perception from control, the method improves interpretability, simplifies debugging, and ensures that the low-level policy always receives physically feasible commands.

Several limitations remain. The robot's arms are fixed, preventing the use of arm motions for balance. Evaluation is currently limited to simulation, and despite including domain randomization and noise in the height maps during training, sim-to-real transfer remains a significant challenge. In particular, the method relies on access to structured terrain height maps, which are difficult to obtain reliably in real-world environments with stairs, steps, or uneven ground. As noted in [12], integrating reliable perception with high-frequency control under such conditions is technically demanding, particularly on computationally constrained platforms.

Future work will focus on developing robust perception pipelines, similar to those in [14], and extending the framework to real hardware. We also plan to integrate the controller into a hierarchical system, where a global planner such as the elastic band method [39] generates base paths while the local controller adapts footsteps. This combination of modularity, robustness, and safety provides a promising foundation for deploying bipedal locomotion in real-world environments.

ACKNOWLEDGMENT

The authors used OpenAI's ChatGPT solely for grammar checking and LaTeX formatting support during manuscript preparation. No AI-generated content was used in the development of the technical or scientific sections of this work.

REFERENCES

- [1] J. Reher and A. D. Ames, "Inverse dynamics control of compliant hybrid zero dynamic walking," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 2040–2047, doi: [10.1109/ICRA48506.2021.9560906](https://doi.org/10.1109/ICRA48506.2021.9560906).
- [2] Y. Gong, R. Hartley, X. Da, A. Hereid, O. Harib, J.-K. Huang, and J. Grizzle, "Feedback control of a Cassie bipedal robot: Walking, standing, and riding a segway," in *Proc. Amer. Control Conf. (ACC)*, Philadelphia, PA, USA, Jul. 2019, pp. 4559–4566, doi: [10.23919/ACC.2019.8814833](https://doi.org/10.23919/ACC.2019.8814833).
- [3] S. Feng, X. Xinjilefu, C. G. Atkeson, and J. Kim, "Robust dynamic walking using online foot step optimization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Daejeon, South Korea, Oct. 2016, pp. 5373–5378, doi: [10.1109/IROS.2016.7759790](https://doi.org/10.1109/IROS.2016.7759790).
- [4] Y. Gong and J. Grizzle, "Angular momentum about the contact point for control of bipedal locomotion: Validation in a LIP-based controller," 2020, *arXiv:2008.10763*.
- [5] A. Del Prete, N. Mansard, O. E. Ramos, O. Stasse, and F. Nori, "Implementing torque control with high-ratio gear boxes and without joint-torque sensors," *Int. J. Humanoid Robot.*, vol. 13, no. 1, Mar. 2016, Art. no. 1550044, doi: [10.1142/s0219843615500449](https://doi.org/10.1142/s0219843615500449).
- [6] J. Shen, "Locomotion analysis and control of a miniature bipedal robot," Ph.D. dissertation, Univ. California, Los Angeles, CA, USA, 2022.
- [7] L. Bao, J. Humphreys, T. Peng, and C. Zhou, "Deep reinforcement learning for bipedal locomotion: A brief survey," 2024, *arXiv:2404.17070*.
- [8] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Vancouver, BC, Canada, Sep. 2017, pp. 23–30, doi: [10.1109/IROS.2017.8202133](https://doi.org/10.1109/IROS.2017.8202133).
- [9] J. Siekmann, Y. Godse, A. Fern, and J. Hurst, "Sim-to-real learning of all common bipedal gaits via periodic reward composition," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, China, May 2021, pp. 7309–7315, doi: [10.1109/ICRA48506.2021.9561814](https://doi.org/10.1109/ICRA48506.2021.9561814).
- [10] Q. Liao, B. Zhang, X. Huang, X. Huang, Z. Li, and K. Sreenath, "Berkeley humanoid: A research platform for learning-based control," 2024, *arXiv:2407.21781*.
- [11] H. Wang, H. Luo, W. Zhang, and H. Chen, "CTS: Concurrent teacher–student reinforcement learning for legged locomotion," *IEEE Robot. Autom. Lett.*, vol. 9, no. 11, pp. 9191–9198, Nov. 2024, doi: [10.1109/LRA.2024.3457379](https://doi.org/10.1109/LRA.2024.3457379).
- [12] J. Siekmann, K. Green, J. Warila, A. Fern, and J. Hurst, "Blind bipedal stair traversal via sim-to-real reinforcement learning," 2021, *arXiv:2105.08328*.
- [13] G. A. Castillo, B. Weng, W. Zhang, and A. Hereid, "Reinforcement learning-based cascade motion policy design for robust 3D bipedal locomotion," *IEEE Access*, vol. 10, pp. 20135–20148, 2022, doi: [10.1109/ACCESS.2022.3151771](https://doi.org/10.1109/ACCESS.2022.3151771).
- [14] H. Duan, B. Pandit, M. S. Gadde, B. Van Marum, J. Dao, C. Kim, and A. Fern, "Learning vision-based bipedal locomotion for challenging terrain," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Yokohama, Japan, May 2024, pp. 56–62, doi: [10.1109/ICRA57147.2024.10611621](https://doi.org/10.1109/ICRA57147.2024.10611621).
- [15] Z. Zhuang, Z. Fu, J. Wang, C. Atkeson, S. Schwertfeger, C. Finn, and H. Zhao, "Robot parkour learning," 2023, *arXiv:2309.05665*.
- [16] T. Haarnoja et al., "Learning agile soccer skills for a bipedal robot with deep reinforcement learning," *Sci. Robot.*, vol. 9, no. 89, p. 8022, Apr. 2024, doi: [10.1126/scirobotics.adi8022](https://doi.org/10.1126/scirobotics.adi8022).
- [17] B. van Marum, M. Sabatelli, and H. Kasaei, "Learning perceptive bipedal locomotion over irregular terrain," 2023, *arXiv:2304.07236*.
- [18] M. Abreu, L. P. Reis, and N. Lau, "Designing a skilled soccer team for RoboCup: Exploring skill-set-primitives through reinforcement learning," *Neural Comput. Appl.*, vol. 37, no. 18, pp. 1–36, Jun. 2025, doi: [10.1007/s00521-025-11151-3](https://doi.org/10.1007/s00521-025-11151-3).
- [19] A. L. Hof, "The 'extrapolated center of mass' concept suggests a simple control of balance in walking," *Hum. Movement Sci.*, vol. 27, no. 1, pp. 112–125, Feb. 2008, doi: [10.1016/j.humov.2007.08.003](https://doi.org/10.1016/j.humov.2007.08.003).
- [20] J. Engelsberger, C. Ott, M. A. Roa, A. Albu-Schäffer, and G. Hirzinger, "Bipedal walking control based on capture point dynamics," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, San Francisco, CA, USA, Sep. 2011, pp. 4420–4427, doi: [10.1109/IROS.2011.6094435](https://doi.org/10.1109/IROS.2011.6094435).
- [21] C. Gaspard, G. Passault, M. Daniel, and O. Ly, "FootstepNet: An efficient actor-critic method for fast on-line bipedal footstep planning and forecasting," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Abu Dhabi, UAE, Oct. 2024, pp. 13749–13756, doi: [10.1109/IROS58592.2024.10802320](https://doi.org/10.1109/IROS58592.2024.10802320).
- [22] H. Duan, A. Malik, J. Dao, A. Saxena, K. Green, J. Siekmann, A. Fern, and J. Hurst, "Sim-to-real learning of footstep-constrained bipedal dynamic walking," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 10428–10434, doi: [10.1109/ICRA46639.2022.9812015](https://doi.org/10.1109/ICRA46639.2022.9812015).
- [23] H. Duan, A. Malik, M. S. Gadde, J. Dao, A. Fern, and J. Hurst, "Learning dynamic bipedal walking across stepping stones," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Mali, Oct. 2022, pp. 6746–6752, doi: [10.1109/IROS47612.2022.9981884](https://doi.org/10.1109/IROS47612.2022.9981884).
- [24] R. P. Singh, M. Benallegue, M. Morisawa, R. Cisneros, and F. Kanehiro, "Learning bipedal walking on planned footsteps for humanoid robots," in *Proc. IEEE-RAS 21st Int. Conf. Humanoid Robots (Humanoids)*, Ginowan, Japan, Nov. 2022, pp. 686–693, doi: [10.1109/HUMANOIDSS3995.2022.10000067](https://doi.org/10.1109/HUMANOIDSS3995.2022.10000067).
- [25] X. B. Peng, G. Berseth, and M. van de Panne, "Terrain-adaptive locomotion skills using deep reinforcement learning," *ACM Trans. Graph.*, vol. 35, no. 4, pp. 1–12, Jul. 2016, doi: [10.1145/2897824.2925881](https://doi.org/10.1145/2897824.2925881).
- [26] X. B. Peng, G. Berseth, K. Yin, and M. Van De Panne, "DeepLoco: Dynamic locomotion skills using hierarchical deep reinforcement learning," *ACM Trans. Graph.*, vol. 36, no. 4, pp. 1–13, Aug. 2017, doi: [10.1145/3072959.3073602](https://doi.org/10.1145/3072959.3073602).

- [27] Z. Xie, H. Y. Ling, N. H. Kim, and M. van de Panne, "ALLSTEPS: Curriculum-driven learning of stepping stone skills," *Comput. Graph. Forum*, vol. 39, no. 8, pp. 213–224, Dec. 2020, doi: [10.1111/cgf.14115](https://doi.org/10.1111/cgf.14115).
- [28] S. Wang, S. Piao, X. Leng, and Z. He, "Learning 3D bipedal walking with planned footsteps and Fourier series periodic gait planning," *Sensors*, vol. 23, no. 4, p. 1873, Feb. 2023, doi: [10.3390/s23041873](https://doi.org/10.3390/s23041873).
- [29] I.-W. Park and J.-Y. Kim, "Fourier series-based walking pattern generation for a biped humanoid robot," in *Proc. 10th IEEE-RAS Int. Conf. Humanoid Robots*, Nashville, TN, USA, Dec. 2010, pp. 461–467, doi: [10.1109/ICHR.2010.5686303](https://doi.org/10.1109/ICHR.2010.5686303).
- [30] H. J. Lee, S. Hong, and S. Kim, "Integrating model-based footstep planning with model-free reinforcement learning for dynamic legged locomotion," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 11248–11255, doi: [10.1109/IROS58592.2024.10801468](https://doi.org/10.1109/IROS58592.2024.10801468).
- [31] W. Wang, S. Liu, Q. Shan, and L. Jia, "Highly dynamic locomotion control of biped robot enhanced by swing arms," 2022, *arXiv:2208.08120*.
- [32] Y. Liu, J. Shen, J. Zhang, X. Zhang, T. Zhu, and D. Hong, "Design and control of a miniature bipedal robot with proprioceptive actuation for dynamic behaviors," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, Philadelphia, PA, USA, May 2022, pp. 8547–8553, doi: [10.1109/ICRA46639.2022.9811790](https://doi.org/10.1109/ICRA46639.2022.9811790).
- [33] J. Siekmann, S. Valluri, J. Dao, L. Bermillo, H. Duan, A. Fern, and J. Hurst, "Learning memory-based control for human-scale bipedal locomotion," 2020, *arXiv:2006.02402*.
- [34] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Sci. Robot.*, vol. 4, no. 26, p. 5872, Jan. 2019, doi: [10.1126/scirobotics.aau5872](https://doi.org/10.1126/scirobotics.aau5872).
- [35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [36] V. Makovychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac gym: High performance GPU-based physics simulation for robot learning," 2021, *arXiv:2108.10470*.
- [37] D. Torricelli, J. Gonzalez-Vargas, J. F. Veneman, K. Mombaur, N. Tsagarakis, A. J. del-Ama, A. Gil-Agudo, J. C. Moreno, and J. L. Pons, "Benchmarking bipedal locomotion: A unified scheme for humanoids, wearable robots, and humans," *IEEE Robot. Autom. Mag.*, vol. 22, no. 3, pp. 103–115, Sep. 2015, doi: [10.1109/MRA.2015.2448278](https://doi.org/10.1109/MRA.2015.2448278).
- [38] H. J. Ralston, "Energy-speed relation and optimal speed during level walking," *Internationale Zeitschrift für Angew. Physiologie Einschliesslich Arbeitsphysiologie*, vol. 17, no. 4, pp. 277–283, Oct. 1958, doi: [10.1007/bf00698754](https://doi.org/10.1007/bf00698754).
- [39] C. Rösmann, W. Feiten, T. Woesch, F. Hoffmann, and T. Bertram, "Trajectory modification considering dynamic constraints of autonomous robots," in *Proc. 7th German Conf. Robot. (ROBOTIK)*, Munich, Germany, May 2012, pp. 1–6.



WILLIAM SULIMAN received the B.Sc. degree in mechatronics engineering and the M.Sc. degree in control and robotics from the Higher Institute for Applied Science and Technology, Damascus, Syria, in 2013 and 2019, respectively. He is currently pursuing the Ph.D. degree in computer science and information technology with Moscow Institute of Physics and Technology (MIPT), Russia.

He is with the Laboratory of Wave Processes and Control Systems, MIPT. His research interests include humanoid robots, reinforcement learning, and optimal control, with a focus on applying learning-based methods to complex locomotion and control tasks in legged robotic systems.



EGOR DAVYDENKO received the M.Sc. degree from Yaroslavl State University, in 2005, and the Ph.D. degree from Vladimir State University, in 2009.

He is currently a Researcher with Moscow Institute of Physics and Technology (MIPT). His work aims to enhance robotic locomotion and perception in dynamic environments by integrating advanced control strategies, computer vision, and engineering principles. He is the author of 15 scientific publications and holds one invention. As part of the Starkit team, he contributed to winning first place in the RoboCup Worldwide Humanoid League, in 2021, along with several other awards. His research interests include robotics, mechatronics, engineering, simulation, and computer vision, with particular emphasis on optimal control and reinforcement learning for legged robots.



EKATERINA CHAIKOVSKAIA received the B.Sc. degree in automation of technological processes and production from Bauman Moscow State Technical University, in 2020, and the M.Sc. degree in engineering systems with a specialization in robotics from Skolkovo Institute of Science and Technology, in 2022.

She is currently a Researcher with Moscow Institute of Physics and Technology (MIPT). She is the author of seven scientific publications in the field of control systems. Her research interests include reinforcement learning for locomotion, legged robotics, and control systems.



ROMAN GORBACHEV received the bachelor's and master's degrees from the Faculty of Radio Engineering and Cybernetics, Moscow Institute of Physics and Technology (MIPT), in 2004 and 2008, respectively, and the Ph.D. degree in technical sciences from MIPT, in 2018.

He is currently the Head of the Laboratory of Wave Processes and Control Systems and the Director of the Research Center for Applied Artificial Intelligence Systems, MIPT. His work aims to advance adaptive control systems and integrate artificial intelligence methods into robotic platforms. He is the author of more than 20 scientific publications, holds five patents for inventions, five utility model patents, and 12 software copyrights. He also coordinates the Starkit team, which won the RoboCup World Championship, in 2021, and is a multiple-time winner of regional championships. His research interests include robotics, systems analysis, multi-agent system optimization, and the development of intelligent control systems, with a focus on collaborative manipulators and humanoid robotics.

...